

《人工智能通识》

第四讲: RAG ——让大模型真正触碰现实世界的第一步

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: 上一讲结尾, 我们把 Prompt 的“边界”画在了哪里?

改 Prompt 能解决的 说不清需求、格式不对、没给例子、没让它一步步想

改 Prompt 解决不了的 模型**根本不知道**的事: 你实验室的数据、最新的文献、内部的规程

上一讲说过: 模型“不知道”的事, 要靠 **RAG** ——先查资料, 再回答。
第二讲互动⑤, 你们为实验室 2000 篇重油加工 PDF 设计过“文献助手”方案, 当时答应**第四讲真的做出来**。

上节课布置的准备: 3 篇你研究方向的 PDF 文献, 带来了吗?
今天它们会被切块、变成向量、存进知识库, 然后被模型“翻着看”来回答你的问题。

情境：问一个模型“没见过”的问题

GPT-4

WA

Meta发布的Llama 2模型能商用吗



截止我最后的知识更新日期（2022年1月），Meta（之前称为Facebook）尚未发布名为“Llama 2”的模型。因此，我不能为您提供关于该模型的商业使用许可或其他具体信息。为了获取最新信息，建议您直接查看Meta的官方公告或相关资源。

模型只记得训练时见过的东西。发布之后发生的事、从没上过网的内部资料，它**一概不知**。

更麻烦的是另一种情况：它不说“不知道”，而是**一本正经地编一个**。

配图：GPT-4 因“知识截止”答不出 Llama 2 的商用许可

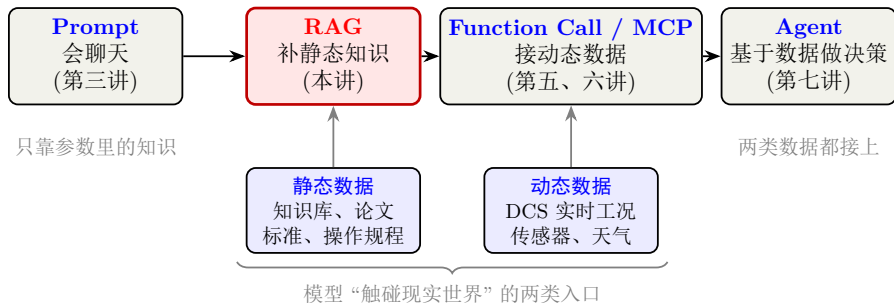
Q:你问过 AI 一个它明显在“编”的问题吗？你当时怎么发现的？

课堂互动① · 举手投票

向 AI 提问“我们实验室去年渣油加氢催化剂失活实验的主要结论是什么”，你猜它会：

A. 老实说不知道 B. 编一段听起来很专业的回答 C. 反问你要资料
先举手，课上用 DeepSeek 现场问一遍验证。1 分钟。

从“会聊天”到“触碰现实世界”：RAG 在路线图上的位置



静态数据 知识库、档案、论文、行业标准、操作规程 $\Rightarrow$ 用 **RAG** 补全 (本讲)

动态数据 天气、传感器读数、DCS 实时工况、交易系统 $\Rightarrow$ 用 **Function Call** / **MCP** 实时调用 (第五、六讲)

两类数据都接上, 模型再基于它们做决策, 就有了 **Agent (智能体)** 的雏形。
RAG 是这条路的第一步。

本讲学习目标

知识目标

说清大模型知识的两个局限与 RAG 解决的问题; 讲出 RAG 的五步流程; 理解分块、向量化、向量数据库、初检与重排各自的作用。

能力目标

能用零代码平台把几篇 PDF 建成知识库并提问; 能为具体的文献 / 规程问答场景设计切块、元数据和评估办法; 能判断一个问题该用 RAG 还是要实时数据。

素养目标

养成“有出处才可信”的习惯: 模型给的引用要点开核对; 私有资料上传前先想清楚合规与保密。

学习建议: 全程跟着“文献助手”这一个案例走, 想想你带来的 3 篇 PDF 在每一步会发生什么。

第一部分 · 为什么需要 RAG: 大模型知识的两个局限

模型的知识来自训练数据, 而训练数据有两个**天生的局限**

记忆是静态的 只记得训练截止前的内容。不知道最新的新闻, 更不知道你实验室数据库里的东西。

知识是有限的 再大的模型也装不下世界上所有细节。行业文档、内部材料、刚发布的法规, 它没见过。

回想第二讲: 知识是在**参数**里的。参数训练完就冻结了, 推理时不再改变。

所以“让它知道新东西”只有两条路: 改参数 (微调, 贵), 或者**把资料放进输入里** (RAG, 便宜)。

“听起来很对, 但其实并不对”: 一段真实感很强的编造

提问: 我们实验室去年做的渣油加氢催化剂失活实验, 主要结论是什么?

回答: 根据贵实验室的研究, 催化剂失活主要由金属沉积 (Ni、V) 和积炭共同导致, 运行 800 小时后脱硫活性下降约 35%, 建议将反应温度提高 $5 \sim 8^{\circ}\text{C}$ 以补偿活性损失……

每个词都像行家说的, 但它**根本没见过你的实验数据**: 800 小时、35% 都是“合理地编”出来的。

知识过期 训练截止后的新文献、新标准, 它不知道

知识缺失 从没上过网的内部报告、实验记录, 它不可能知道

记忆模糊 见过但记不清, 两篇文献的数据混在一起说

Q: 这三种情况里, 哪一种最危险? 为什么“记忆模糊”往往比“不知道”更难发现?

“它不是能联网搜索了吗？”——搜索不等于一切

联网搜索能解决的

- ▶ 公开、最新的信息：新品发布时间、新闻动态
- ▶ 早期模型因“知识截止”只能说“我不知道”，今天的产品型模型已能搜到再答

联网搜索解决不了的

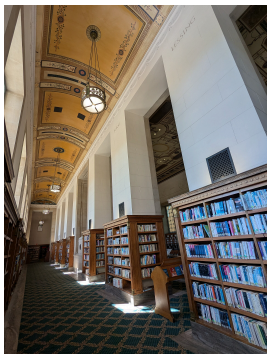
- ▶ **垂直领域**：法律条文解读、医学病例分析、交通控制参数
- ▶ **不在公网上**的东西：你的实验记录、装置的操作规程、课题组的未发表数据

化工版例子：“我们这套催化裂化装置提升管出口温度的控制范围是多少？”

答案写在**本厂的工艺卡片**里，搜遍全网也搜不到——只能**把工艺卡片交给模型**。

这时候，就轮到 RAG 出场了。

RAG: Retrieval-Augmented Generation, 检索增强生成



先去翻, 再作答

通俗地说: 给大模型装一台“**搜索引擎外挂**”。

回答之前, 先去数据库或文档库里翻一翻, 把相关内容找出来, 再结合自己的生成能力组织答案。

Retrieval 检索 找到相关资料

Augmented 增强 把资料加进输入, 增强上下文

Generation 生成 模型基于资料写出回答

模型的语言理解能力 + 外部资料的事实 = 更准确的回答。

第二讲说过的类比现在更具体了: 闭卷考试 → **开卷考试**, 而且开的是**你指定的那本书**。

RAG 在化工场景里能干什么？都是“静态资料”问题

场景	知识库放什么	学生 / 工程师问什么
文献综述	课题组 2000 篇重油加工 PDF	近三年渣油加氢催化剂失活的结论有哪些？
实验 SOP	高压釜、色谱、热重的操作规程	高压釜升温前要检查哪几项？依据是哪一条？
安全规程	危化品 MSDS、应急预案	四氢呋喃泄漏后先做什么？
工艺文件	装置操作手册、工艺卡片	常压塔顶回流量的正常区间是多少？
物性数据	手册、内部测定的物性表	这批减压渣油的运动粘度是多少？

共同点：答案[已经写在某份文件里](#)，只是模型没读过。RAG 的任务就是替它找到那一页。

反例：“三号反应器现在的床层温度是多少？”答案在 DCS 实时数据里，不在任何文件里——那是第五讲的事。

判断题: 这个问题该不该用 RAG? (先举手, 再揭晓)

课堂互动②

1. “请解释什么是催化裂化。”
2. “我们实验室 2024 年 3 月那批催化剂的比表面积测了多少?”
3. “现在 2 号精馏塔的塔顶压力是多少?”
4. “GB 30871 里关于受限空间作业的气体检测要求是什么?”
5. “帮我把这段英文摘要翻译成中文。”

1. ✗ 通用知识, 模型本来就会, 直接问即可 (改 Prompt 就够)。
2. ✓ 内部实验记录, 模型不可能知道, 要把记录放进知识库。
3. ✗ 实时数据, 不在任何文件里, RAG 查不到——要 Function Call 去读 DCS。
4. ✓ 标准条文要求逐字准确, 把标准原文放进知识库, 回答带条款号。
5. ✗ 纯语言能力任务, 不需要外部知识。

判断标准一句话: 答案是否写在某份模型没读过的文件里。

第一部分小结: RAG 解决的是“静态知识”问题

模型的参数化知识是**封闭的**: 覆盖不了企业私有资料、法规细则和长尾专业内容。

RAG 通过检索外部知识, 把文档、资料、规范**补充给模型**, 让回答更贴近真实世界。

Prompt

告诉模型该怎么做

第三讲

RAG

给模型它没有的资料

本讲

Function Call / MCP

让模型实时取数据、做操作

第五、六讲

微调

改参数, 改变模型本身

第九讲

Q:“先翻资料再回答”听起来很简单, 那系统内部到底发生了什么? ⇒ 第二部分

第二部分 · RAG 的工作原理：五步走一遍

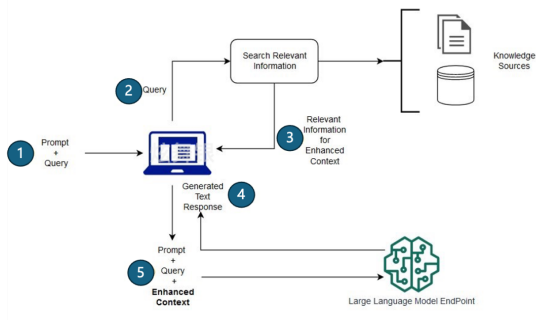


Image From AWS: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>

图里有四个角色:

- ▶ **用户**: 输入 Prompt 和 Query
- ▶ **检索模块**: 去知识源里找
- ▶ **知识源**: 文档、数据库
- ▶ **大模型端点**: 最后写回答

注意: 大模型**最后**才出场, 而且拿到的输入比用户原话**多了一大块**。

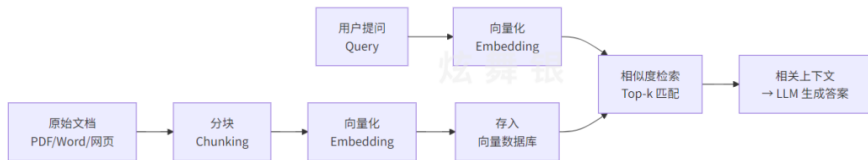
配图 (来自 AWS): RAG 运行时的五个步骤

五步，每一步在做什么

	步骤	文献助手里的样子
①	Prompt + Query	老师输入：“近三年关于渣油加氢催化剂失活的结论有哪些？”
②	Query → 搜索相关信息	系统把问题交给检索模块，到 2000 篇 PDF 建成的知识源里找
③	返回相关信息	找回最相关的几段原文，作为“增强上下文”
④	Prompt + Query + 增强上下文 → 大模型	把问题和这几段原文拼在一起，一起发给模型
⑤	生成文本响应	模型读着原文写回答，并标出每条结论来自哪篇

用第二讲的词说：步骤②③靠 **embedding 与向量距离**；步骤④受 **上下文长度** 限制；
步骤④拼出来的那段话，就是第三讲讲的一条精心组织的 **Prompt**。

重点不在“生成”，而在“建库”和“匹配”

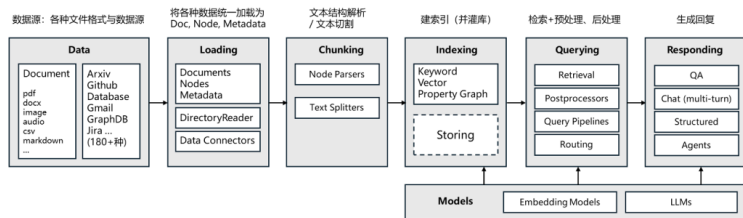


配图：上面一行是建库（离线做一次），下面一行是查询（每次提问都跑）

RAG 的关键环节是**知识库建设**和**检索匹配**：
知识源建得好不好、Query 能不能和知识源精准对上，决定了最终生成的效果。

模型再强，拿到的是错的段落，也只能“一本正经地”基于错段落作答。
垃圾进，垃圾出。

工程上的端到端视角: LlamaIndex 的数据流



配图: LlamaIndex 把整条链路拆成 Data / Loading / Chunking / Indexing / Querying / Responding

Data → Loading

各种格式 (PDF、Word、网页、数据库) 统一读进来, 附上元数据

Chunking

文本切分成片段

Indexing → Storing

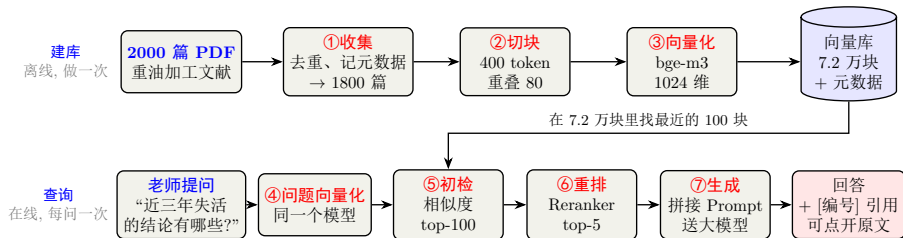
向量化、建索引、存进向量库

Querying → Responding

检索、重排、路由; 问答、多轮对话、结构化输出

第三、四部分沿着这条线, 用文献助手把每一格都填上。

贯穿案例：把 2000 篇重油加工 PDF 做成文献助手



上面一行**建库**只做一次 (新文献来了再增量加); 下面一行**查询**每提一个问题都跑一遍。

第二讲互动⑤留下的三个问题, 今天逐个回答:

2000 篇 PDF 怎么变成向量库? · 检索出 20 段会不会超上下文, 怎么取舍? · 怎样让老师能点开原文核对?

第一步：文档收集——把分散的资料汇到一个统一入口

第三部分 · 建库：文档收集 → 分块 → 向量化 → 存储

把散在各处、各种格式的资料聚起来，但**不是越多越好**：零碎资料不加选择地塞进去，会增加成本，还会在检索时带来**噪音**。

围绕问题收 文献助手要回答的是“重油加工”的问题，课题组顺手存的机器学习入门教程就别放

先去重 同一篇文献的预印本、正式版、补充材料，留一份主文本

顺手记元数据 题目、作者、年份、期刊、DOI、所属方向 (加氢 / 催化裂化 / 焦化)，后面过滤和溯源全靠它

注意解析质量 扫描版 PDF 要先 OCR; 表格、公式抽出来往往是乱码，要检查

案例：2000 篇 PDF 去重后约 1800 篇，每篇标上年份和方向——这一步没有算法，全是耐心。

第二步: 分块 (Chunking) —— 为什么要把文档切碎?

一篇 PDF 全文 (约 16000 token): 一次塞不进上下文, 也没法和一个具体问题精准匹配



切分 (Chunking): 每块 400 token, 相邻块重叠 80 token



块大小 400 token

重叠 80 token

答案若正好落在切点上, 重叠区能把它完整留在其中一块里

每一块之后单独向量化、单独参与检索; 切点尽量落在段落 / 句子边界

两个原因: 模型的**上下文长度**有限, 一次塞不下整篇; 检索也需要**合适大小**的**片段**来和问题匹配。

所以要把长文档切成较小的片段 (Chunk), 并让相邻片段**部分重叠** (overlap), 避免答案正好被切断。

切多大？在“粒度”和“上下文完整”之间找平衡

	会出什么问题	文献助手里的表现
粒度过大	命中范围太宽，混进很多不相关内容，检索精准度下降	一整节“结果与讨论”当一块，问失活机理却连带把动力学拟合全塞进来
粒度过小	检索更细，但关键信息被拆散，模型缺乏完整上下文	一句话一块，“活性下降 35%”和“运行 800 h 后”分在两块，拼不回来
跨片段	答案横跨两块甚至多块，切得太碎就拼不出完整答案	表格的表头和数据行被切开

最佳实践：按**一定粒度**切分，并设置**部分重叠**。
常见起点：每块 300 ~ 500 token，重叠 50 ~ 80 token；尽量在段落、句子边界处切。

数字只是经验起点，不同文档类型（论文 / 规程 / 表格）最合适的粒度不同，要靠评估调。

动手：给这段文献切块

课堂互动③

下面是一篇渣油加氢文献“结果与讨论”的节选 (约 130 字)。4 人一组, 3 分钟, 用笔画出切点, 写出你们的块大小和重叠量, 并说明为什么这样切。

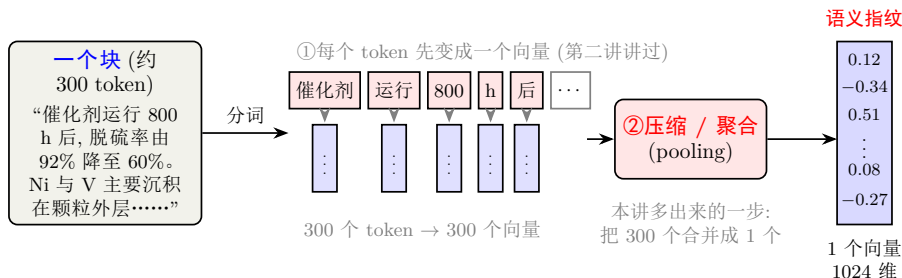
“催化剂运行 800 h 后, 脱硫率由 92% 降至 60%。表 3 给出了各阶段的金属沉积量。Ni 与 V 主要沉积在颗粒外层 50 μm 内, 形成硫化物壳层, 堵塞孔口。相比之下, 积炭在前 100 h 内迅速累积, 之后趋于平稳。因此, 前期失活以积炭为主, 后期以金属沉积为主。”

然后各组互相提问, 看对方的切法能不能答上来:

- ▶ “运行 800 h 后脱硫率降到多少?” 数字和时间在同一块里吗?
- ▶ “前期和后期的失活原因分别是什么?” 结论句和证据句还在一起吗?
- ▶ “表 3 是什么内容?” “表 3” 三个字和表格本身能对上吗?

切块没有标准答案, 只有“针对这些问题好不好用”。 这就是为什么后面一定要评估。

第三步：向量化 (Embedding) ——给每一块算一个“语义指纹”



和第二讲讲的一致：先把块里的每个 token 变成向量；不同的是，这里**再多做一步压缩 / 聚合**，把 300 个 token 的信息合并成一个**固定维度的向量**（比如 768 或 1536 维）。不管块有多长，出来的都是同一长度的一个向量——这就是这段文字的**语义指纹**。

整份知识库向量化之后, 是一个 $N \times M$ 的矩阵

 N 文档被切成的片段总数

M 向量维度, 由 embedding 模型决定 (768 / 1024 / 1536)

文献助手算一笔账：

1800 篇 $\times$ 每篇约 40 块

$$N \approx 7.2 \text{ 万块}$$

每块 400 token, 全部送 embedding 模型 约 2900 万 token

embedding 按 0.5 元 / 百万 token 估 一次性成本约 **15 元**

得到的矩阵

72000 × 1024 个浮点数, 约 300 MB

价格与维度为示意; 用 hf-mirror 下载开源模型 (如 BAAI/bge-m3) 在本地跑, 则不花钱只花时间。

Q:这 7.2 万个向量存在哪、怎么找? Excel 存得下,但一行行比对 7.2 万次会有多慢?

第四步：存进向量数据库——先用“二维表格”类比

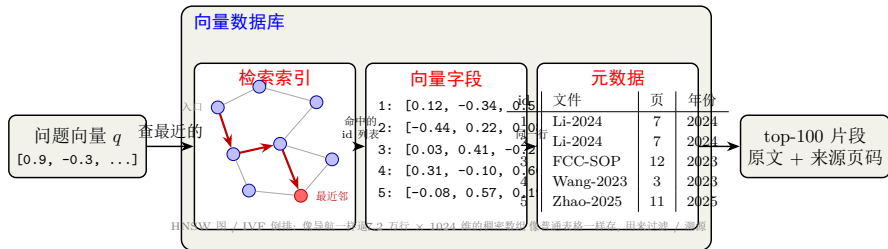
id	向量 (embedding, 1536维)	原始文本片段	来源文档	时间	标签
1	[0.12, -0.34, 0.56, ..., 0.91]	“如何重置密码”	FAQ.doc	2023-09	客服
2	[-0.44, 0.22, 0.18, ..., -0.07]	“退货需要什么条件？”	Policy.pdf	2024-01	售后
3	[0.03, 0.41, -0.62, ..., 0.25]	“去年机械维修花了多少钱”	Report.xlsx	2024-06	财务

配图: 用表格类比向量数据库, 每行 = 一个块的向量 + 原文 + 元数据

id	向量 (1024 维)	原始文本片段	来源文档	年份	方向
1	[0.12, -0.34, ...]	“催化剂运行 800 h 后, 脱硫率由...”	Li-2024.pdf p.7	2024	渣油加氢
2	[-0.44, 0.22, ...]	“Ni 与 V 主要沉积在颗粒外层...”	Li-2024.pdf p.7	2024	渣油加氢
3	[0.03, 0.41, ...]	“提升管出口温度控制在...”	FCC-SOP.pdf p.12	2023	催化裂化

传统数据库靠**主键**快速找一行; 向量数据库靠**语义相似度**找最接近的几行。

真实的向量数据库比表格复杂：三个部分配合工作



几毫秒完成：靠索引找到最近的 id，再顺着 id 取出向量、原文和元数据

向量字段 不是一列文字，而是一组组密密麻麻的数字 (稠密数组)，把语义压缩成坐标点

检索索引 不是一行行去找，而是用“地图 / 导航算法” (HNSW 图索引、IVF 倒排索引) 在海量向量里快速找到最近的点

元数据 来源、时间、类别，像表格那样保存，用来过滤或追溯

不理解索引算法没关系，只要知道：7.2 万个向量里找最近的 100 个，靠索引只要几毫秒。

别小看元数据：过滤和溯源都靠它

过滤 (检索前缩小范围)

- ▶ “近三年的结论” $\Rightarrow$ 先按 年份 ≥ 2023 过滤, 再算相似度
- ▶ “只看加氢方向” $\Rightarrow$ 按 方向 = 渣油加氢过滤
- ▶ 工厂版: 按**装置编号**、**文件版本**过滤, 别把老版本操作规程检出来

溯源 (回答后能点开核对)

- ▶ 每个块都带 文件名 + 页码 + 章节
- ▶ 模型回答时引用块编号, 系统换成 “Li-2024.pdf 第 7 页”
- ▶ 老师点开那一页, 30 秒就能判断模型有没有说错

文档收集时多花 10 分钟记元数据, 查询时省下的是**每一次**核对的时间。

很多 RAG 系统效果差, 根源不在模型, 而在建库时元数据没记全。

第三部分小结：库建好了——第二讲互动⑤的第一个问题有了答案

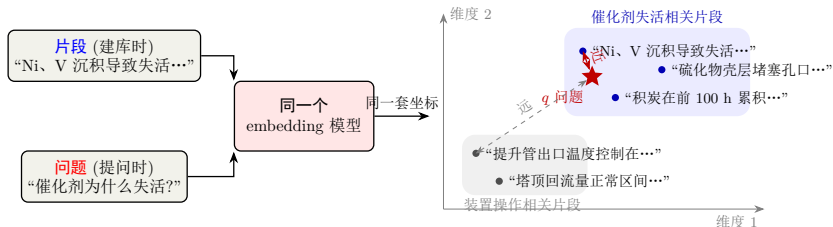
步骤	做什么	文献助手的参数
文档收集	围绕问题收，去重，记元数据	1800 篇，带年份 / 方向 / DOI
分块	切成片段，相邻重叠	400 token, 重叠 80, 按段落边界
向量化 存储	每块一个固定维度向量 向量 + 原文 + 元数据，建索引	bge-m3, 1024 维, 约 7.2 万块 向量库 (FAISS / Milvus 等), HNSW

“2000 篇 PDF 怎么变成向量库?” $\Rightarrow$ 收集 $\rightarrow$ 切块 $\rightarrow$ embedding $\rightarrow$ 存进向量库，一次性做完，之后新文献来了再增量加。

Q:老师现在提了一个问题，系统怎么在 7.2 万块里找到最相关的那几块?
 $\Rightarrow$ 第四部分

第一步: 用户提问 → 向量化 (与片段同一语义空间)

第四部分 · 查询: 提问向量化 → 初检 → 重排 → 生成答案



用户的问题先被分词, 再通过**和处理片段时相同的 embedding 模型**转成一个向量。

这样问题和片段就在同一个空间里, 只需比较它们的“距离”, 就能判断哪些片段最相关。

建库用 A 模型、提问用 B 模型, 两套坐标系对不上——这是新手最常见的错误之一。

“距离” 怎么算？最常用的是余弦相似度

$$\cos(\theta) = \frac{q \cdot c}{\|q\| \|c\|}$$

取值 $-1 \sim 1$, 越接近 1 越相似 (只看方向, 不看长短)

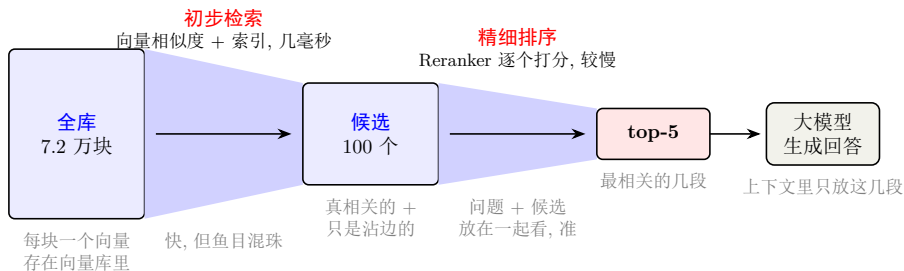
用二维玩具例子感受一下 (真实向量有上千维): 问题“催化剂为什么失活” $\rightarrow q = (0.9, 0.4)$

候选片段	向量	与 q 的余弦	
片段 1: “Ni、V 沉积导致失活…”	(0.8, 0.5)	0.99	✓ 最近
片段 2: “积炭在前 100 h 累积…”	(0.7, 0.6)	0.97	✓ 次近
片段 3: “提升管出口温度控制在…”	(0.1, 0.9)	0.49	✗ 不相关

检索模块做的事就是: 算出问题向量和每个候选块的相似度, 按大小排序, 取前几个。

也有用欧氏距离、内积的; 向量归一化后, 内积 = 余弦。

检索不是一次搞定：初步检索 + 精细排序



初步检索 在向量空间里找最近邻，快速捞出一批候选（比如 100 个）。速度快，但**鱼目混珠**：有的很相关，有的只是沾边。

精细排序 用一个更聪明的“二次筛选器”——重排序器 (Reranker)，把“问题 + 每个候选”逐个放进模型打分，重新排列。

最终选择 按新分数取最优的几个 (top-k, 比如前 5 个)，交给大模型生成答案。

Reranker 为什么打分更准？因为它把问题和片段“放在一起看”

	初检 (向量相似度)	重排 (Reranker)
怎么算	问题和片段各自算向量, 再比距离	问题 + 片段拼在一起送进模型, 直接输出相关度分数
能看到什么	各自的“大意”	两者之间逐词的对应关系和上下文
速度	极快, 靠索引, 7.2 万块里找 100 个只要几毫秒	慢, 每个候选都要跑一遍模型
用在哪	全库粗筛	只对 100 个候选精筛

重排器通常基于更强的语言模型 (如 BERT、RoBERTa 一类), 能理解更深层的语义。

类比: 初检像按关键词翻文献目录, 重排像把候选的 100 篇摘要逐篇读一遍再定。

文献助手里的选择: 初检 top-100 → 重排 top-5。没有重排也能跑, 只是“沾边”的段落更多。

最后一步：拼接输入，交给大模型生成答案

[系统] 你是重质油加工方向的文献助手。只根据下面提供的资料回答，每条结论后用 [编号] 标出来源；资料里没有的内容，明确说“资料中未提及”，不要猜。

[资料 1] (Li-2024.pdf, p.7) 催化剂运行 800 h 后，脱硫率由 92% 降至 60%

[资料 2] (Wang-2023.pdf, p.3) 积炭在前 100 h 内迅速累积，之后趋于平稳.....

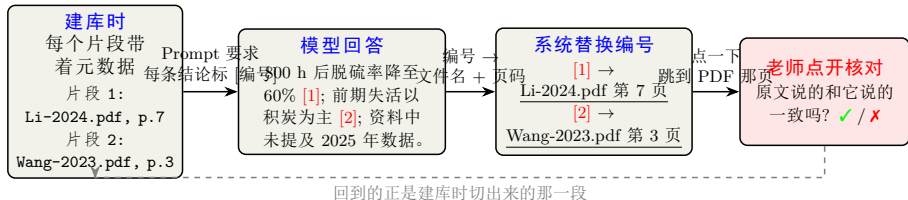
[资料 3] (Zhao-2025.pdf, p.11) 采用大孔保护剂后，金属容量提高约 40%

[问题] 近三年关于渣油加氢催化剂失活的结论有哪些？

拼接输入：问题 (Query) + 检索到的片段 (Context) 组成新 Prompt → **送入大模型** (DeepSeek、Qwen 等都行) → **生成**完整自然的回答，而不是原文的简单拼接。

认出来了吗？这就是第三讲的**角色 + 指令 + 输入 + 输出格式**，只是“输入”这一栏由系统自动填。

引用溯源：让老师能点开原文核对——第二讲互动⑤的第三个问题



做法很朴素：每个片段建库时就带着**文件名 + 页码**；Prompt 里要求模型**每条结论后标 [编号]**；

系统把 [编号] 换成可点击的链接，直接跳到 PDF 的那一页。

有出处，才能核对；能核对，才敢用。 没有引用的 RAG 回答，和开场那段“编造”没有本质区别。

案例分析：检索出 20 段，超上下文了怎么办？

课堂互动④

老师问“近三年渣油加氢催化剂失活的结论有哪些”，初检重排后有 20 个片段分数都很高，每段 400 token，加起来 8000 token；你们用的模型上下文只有 8K，还要留位置给系统提示和回答。

4 人一组，4 分钟，给出取舍方案，写在纸上；选 2 组说，其他组补充。

可以考虑的方向（不止这些）：

- ▶ 只取 top- k ($k = 5$)：简单，但会不会漏掉某篇的关键结论？
- ▶ 按来源去重：同一篇文献只留分数最高的 1 ~ 2 段，保证覆盖更多文献
- ▶ 按年份分组：2023 / 2024 / 2025 各取几段，回答自然按时间线组织
- ▶ 分两轮：先让模型给每段写一句摘要（输出短、便宜），再把 20 句摘要一起送去总结
- ▶ 换上下文更长的模型：128K 的模型能一次装下，但成本和“注意力稀释”呢？（第二讲）

顺带一问：如果模型回答“资料中未提及 2025 年的结论”，这是好事还是坏事？

怎么知道这个文献助手“好不好用”？三种失败，分开测

失败模式	现象	怎么发现 / 怎么改
没检测到	答案明明在库里，却没被捞出来，模型说“未提及”	准备 30 个已知答案在哪的问题，看正确片段是否进了 top-5 (命中率)；改切块、换 embedding 模型、加重排
检错了	捞出来的是沾边的段落，模型基于错段落作答	看 top-5 里有几个真相关 (精确率)；加元数据过滤、调 top-k
答歪了	片段是对的，模型却加了资料里没有的话	逐句对照引用 (忠实度)；收紧系统提示，要求“只根据资料”

评估的顺序：先测检索，再测生成。**检索没对，调 Prompt 没用**；检索对了，生成问题才轮到 Prompt。

30 个问题的“考卷”由课题组自己出，每次改参数都重考一遍——和做实验要有对照组是一个道理。

第四部分小结：一次提问，系统里发生的五件事

	步骤	关键点
1	问题向量化	用和建库相同的 embedding 模型
2	初步检索	向量相似度 + 索引, 快, 取 top-100
3	重排序	Reranker 把问题和片段一起看, 准, 取 top-5
4	拼接生成	系统提示 + 片段 + 问题, 要求带引用
5	溯源核对	[编号] → 文件名 + 页码, 人来把关

第二讲互动⑤的三个问题, 现在都有了答案:

建库四步 · 取舍靠 top- k / 去重 / 分组 / 两轮 · 溯源靠元数据 + 引用编号

Q: 原理都懂了, 真要动手, 需要写多少代码? ⇒ 第五部分: 可能比你想的少得多

第五部分 · 动手搭一个文献助手: 零代码平台与十几行代码

环节看着多, 但工程上早已**工具化**

“数据接入 → 切分 → 向量化 → 建索引 / 存储 → 检索 → (可选) 重排 → 生成” 这条链路, 有现成的方案帮你拼好。

	零代码平台	Python 框架
代表	阿里云百炼知识库、Dify	LlamaIndex、LangChain
上手	上传文件, 点几下, 10 分钟	装环境, 写十几行, 1 小时
能看到什么	界面上直接看到切好的片段和命中结果	每一步的参数都能改, 能接自己的向量库
适合谁	今天的你: 先跑通, 建立手感	想把它做成课题组正式工具的你

先用平台把 3 篇 PDF 跑通, 再看代码里每一行对应今天的哪一步。

零代码平台的设置页，其实就是今天讲的每一步



界面上的设置 (示例值)

分段标识符: 空行

分段最大长度: 1024 字符

分段重叠长度: 50 字符

索引方式: 高质量

Embedding 模型: text-embedding-3-large

检索: 混合 (语义 0.7 / 关键词 0.3)

Rerank 模型: 可选开启

Top K: 3

Score 阈值: 0.5

对应今天的

按段落边界切块大小

overlap
向量化 (用 embedding 模型)
语义指纹

初检

重排
送给模型几段
太不相关的不要

Q:混合检索里的“关键词”一路是干什么的？什么问题靠语义找不到、靠关键词一下就中？

现场演示: 把你带来的 3 篇 PDF 做成知识库

课堂互动⑤

打开 [阿里云百炼控制台](#) (步骤见 [百炼文档](#)“知识库” 章节), 全班跟着做, 10 分钟:

- ①新建知识库, 上传 3 篇 PDF, 默认切分, 等索引完成;
- ②新建“应用”, 挂上知识库, 勾选“显示引用”;
- ③问一个只有这 3 篇文献能回答的问题, 如“第二篇文献用的催化剂载体是什么? 负载量多少?”;
- ④同一个问题不挂知识库再问一遍 (或去 [DeepSeek](#) 问), 对比两个回答。

观察并记下三件事:

1. 平台把你的 PDF 切成了多少块? 点开看两块, 切在了合理的地方吗? 表格和公式还完整吗?
2. 回答里的引用点开, 是不是真的指向了那一段? 有没有“引用是对的, 结论却说歪了”的情况?
3. 不挂知识库时, 模型是老实说不知道, 还是编了一个? ——回到开场的投票, 你猜对了吗?

没带 PDF 的同学, 用 [Kimi](#) 或 [腾讯元宝](#) 的“上传文件后提问”也能体验, 只是看不到切块和参数。

换成代码: LlamaIndex 把整条链路写成了十个步骤



	代码里的步骤	对应今天的
①	导入	—
②	读入 ./data 目录下的文档	文档收集
③	SentenceSplitter(500, 80)	分块 + 重叠
④	选 BAAI/bge-m3 向量模型	向量化
⑤	准备 FAISS 向量库	存储
⑥	建索引 (向量化 + 写入)	建库完成
⑦	持久化到磁盘	下次直接加载
⑧	检索 top-8, 重排 top-5	初检, 重排
⑨	选一个大模型	生成
⑩	提问, 打印答案和出处	溯源

配图: 完整代码截图。向量模型从 [hf-mirror.com](https://huggingface.co/BAAI/bge-m3) 下载, 本地运行, 不需要 API Key。

真正“干活”的只有这几行

```
②读 PDF: docs = SimpleDirectoryReader("./data").load_data()
③切块: splitter = SentenceSplitter(chunk_size=500, chunk_overlap=80)
④向量模型: embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-m3")
⑥建库: index = VectorStoreIndex.from_documents(docs,
        embed_model=embed_model, transformations=[splitter])
⑧初检: retriever = index.as_retriever(similarity_top_k=8)
⑧重排: reranker = SentenceTransformerRerank(
        model="BAAI/bge-reranker-base", top_n=5)
⑩提问: question = " 近三年渣油加氢催化剂失活的结论有哪些? 给出处。"
        response = query_engine.query(question)
⑩      出      处: for n in response.source_nodes: print(n.score,
n.metadata["file_name"])
```

改成文献助手 把 ./data 换成 2000 篇 PDF 的目录; 问题换成“近三年渣油加氢催化剂失活的结论”

改参数在哪 块大小、重叠、top- k 、重排 top- n 都在这几行里, 和平台设置页一一对应

大模型接谁 本地 Ollama 跑开源模型, 或接 DeepSeek / 百炼 API, 一行切换 (完整示例见 [LlamaIndex 仓库](#))

兑现承诺：第二讲互动⑤的方案，今天的完整版

环节	第二讲时你们写的	今天可以写成的
建库	“切块、embedding、存起来”	去重到 1800 篇，记年份 / 方向 / DOI; 400 token 块、重叠 80; bge-m3 向量化; 存 FAISS 或 Milvus 并建 HNSW 索引
检索	“找最相关的”	元数据先过滤“近三年”; 初检 top-100; Reranker 重排取 top-5; 同一篇最多留 2 段
生成	“一起给模型”	系统提示: 只根据资料、逐条带 [编号]、没有就说没有; 接 DeepSeek 或 Qwen
溯源	“让老师能点开原文”	[编号] → 文件名 + 页码 → 直接跳到 PDF 那一页
评估	(当时没想到)	课题组出 30 道“已知答案”的题, 测命中率、精确率、忠实度, 每次改参数重考

多出来的“评估”一行最重要: 没有它, 你永远不知道改块大小是变好了还是变坏了。

跑通之后, 效果不好? 常见的坑与对策

现象	多半是	试试
问具体数字, 答的是笼统描述	块太大, 数字被淹没	块缩到 300 token; 或“父子分段”: 小块检索, 大块作上下文
问“表 3”, 答非所问	PDF 表格解析成了乱码	换解析器; 表格单独抽出转成文字描述再入库
老规程和新规程混着答	元数据没记版本	记文件版本 / 生效日期, 检索前过滤
问“GB 30871”这种编号找不到	纯语义检索对编号、型号不敏感	开混合检索, 加关键词一路
引用是对的, 结论却多了一句	模型“发挥”了	收紧系统提示; 降低 temperature; 人核对
新文献加进去后没被检到	忘了重建 / 增量更新索引	建立“新文献入库”的固定流程

每条对策都对应今天的某一步——调 RAG 的过程, 就是把五步逐个排查一遍。

有出处才可信: 用 RAG 时的三条底线

- ▶ **引用要点开核对**: RAG 给了页码不等于说对了, “引用正确、结论说歪”是最隐蔽的错误。写进论文、报告的每个数字, 回到原文确认一次。
- ▶ **私有资料上传前先想清楚**: 课题组未发表的数据、企业的工艺卡片和操作规程属于保密信息, 不要随手传到公有平台的知识库; 要用就用本地部署或单位内网方案, 并征得数据所有者同意。
- ▶ **综述是你写的, 不是它写的**: 让文献助手帮你找到 20 篇相关文献是效率; 把它生成的段落原样贴进综述而不核对、不标注, 是学术不端。

Q:讨论 (2 分钟): 你带来的 3 篇 PDF 里, 有没有哪一篇是不该传到公有平台上的? 判断依据是什么?

把“能核对”当成 RAG 的一部分来设计, 而不是事后补救。

小组讨论: 为你们的研究方向设计一个知识库

课堂互动⑥

4 人一组, 6 分钟, 在一页纸上写出方案, 五栏都要填; 选 2 组上台讲 1 分钟, 其他组按右侧标准打分。

方向举例: 催化裂化催化剂、渣油加氢、生物质热解、CO₂ 捕集、化工安全——用你自己的。

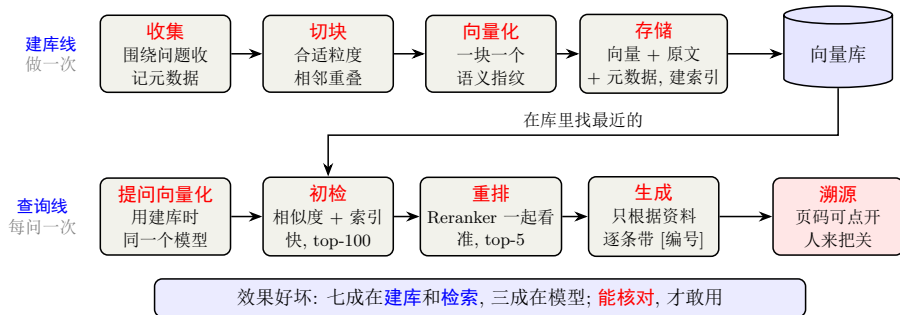
五栏:

1. **收什么**: 哪些文档进库, 哪些不进, 为什么
2. **怎么切**: 论文 / 规程 / 表格各用多大的块
3. **记什么元数据**: 至少 3 个字段, 各用来过滤还是溯源
4. **怎么评估**: 出 5 道“已知答案”的题写在纸上
5. **合规**: 哪些文档不能上公有平台, 怎么处理

评分标准 (每项 0 ~ 2 分):

- ▶ 文档范围是否围绕一类明确的问题
- ▶ 切块粒度是否针对文档类型区分
- ▶ 元数据是否真能用来过滤 / 溯源
- ▶ 评估题是否能区分“没检测到”和“答歪了”
- ▶ 合规判断是否有依据

本讲总结: RAG 的两条线、九个环节



建库线做一次: 收集 → 切块 → 向量化 → 存储; 查询线每问一次: 向量化 → 初检 → 重排 → 生成 → 溯源。

效果好坏, 七成在**建库和检索**, 三成在**模型**; 而“能不能核对”, 决定了你敢不敢用。

随堂快测

课堂互动 · 3 道题

- Q1 (单选) 文献助手回答“资料中未提及”，但答案其实在库里的某一段。最该先检查的是：
A. 换一个更大的生成模型 B. 检索是否把那一段捞进了 top-k
C. 把系统提示写长一点 D. 提高 temperature
- Q2 (判断) 建库时用 A 模型做 embedding, 提问时换成更新的 B 模型做 embedding, 检索效果会更好。
- Q3 (简答) “2 号精馏塔现在的塔顶压力是多少？”为什么 RAG 回答不了？用今天的话说清楚，并指出它需要什么。

答案: Q1 选 **B**, 先测检索再测生成; Q2 **✗** 错误, 问题和片段必须在同一个语义空间, 换模型要全库重建;
Q3 这是实时的动态数据, 不写在任何文件里, RAG 只能补静态知识; 需要 Function Call 去读 DCS (下一讲)。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 建库并出考卷: 在 [阿里云百炼](#) 用你的 3 篇 PDF 建知识库, 自己出 5 道“已知答案在第几页”的问题, 记录每题是否命中、引用是否指对、回答有没有多说, 做成一张表 (含截图)。
- 必做② 改一个参数: 把分段长度改成原来的一半 (或两倍), 重考同一份 5 道题, 写 150 字说明哪些题变好、哪些变坏、为什么。
- 选做③ 跑通代码: 从 [hf-mirror.com](#) 下载 bge-m3, 用第 41 页那几行 LlamaIndex 代码在本机跑通, 把打印出的“相似度 + 来源”截图交上来。

拓展阅读: [阿里云百炼文档](#) 的知识库 / RAG 章节 (中文, 有截图);
提出 RAG 的论文 *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020, 选读)。

下讲预告: Function Call —— 让大模型实时感知与交互



中控室的数据每秒都在变,
RAG 追不上

RAG 的边界

静态资料能补, 实时数据补不了

模型怎么“打电话”

它不执行, 只说“该调哪个函数、传什么参数”

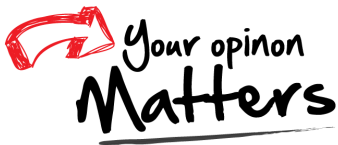
读 DCS、查天气

函数定义、参数 JSON、结果回填

和 RAG 怎么配合

查规程用 RAG, 读实时值用 Function Call

Q: 课前思考: 找一个你希望 AI 能“实时查到”的数据 (设备读数、实验室温湿度、某个网站的数值), 想想它从哪来、怎么取。



第四讲 · RAG —— 让大模型真正触碰现实世界的第一步

周天航 · 中国石油大学 (北京)

下课。请完成课后任务，下次课带一个你想让 AI “实时查到” 的数据来。